Descarte Aqui — arquitetura do sistema IoT

ESP32-WROOM-32 como servidor HTTP · sensores e atuadores locais · painel web e modelo 3D sincronizados com a placa

HARDWARE

SENSORES

HC-SR04
Presença do objeto (ultrassom)
`TRIG 19 · ECHO 18 · limite 5 cm`

Sensor indutivo
Metal / não-metal (pilha?)
`GPIO 32 · INPUT_PULLUP · LOW = metal`

HX711 + célula de carga
Peso da pilha separada
`DT 16 · SCK 4`

ATUADORES E DISPLAY

Servo de separação
Inclina a bandeja para um dos lados
`GPIO 25 · 20° / 90° / 160°`

Servo da portinha
Libera a retirada do não-metal
`GPIO 26 · 0° fechada / 90° aberta`

OLED SSD1306
Mensagem para quem está na frente
`I2C · SDA 21 · SCL 22 · 0x3C`

LIMITAÇÃO CONHECIDA

`FATOR_CALIBRACAO = 420.0` é um valor provisório. O campo `peso_g` só ganha significado físico depois de calibrar a célula com um peso conhecido (README §8).

leitura

comando

FIRMWARE — src/main.cpp

`loop()`
1. server.handleClient() — atende o navegador
2. reconecta o Wi-Fi se tiver caído
3. lê distância, metal e peso a cada ~300 ms

CICLO AUTOMÁTICO (máquina de estados)



`aguardar(ms)`
Substitui o delay(). Durante a espera do ciclo segue chamando server.handleClient(), então a página não congela enquanto o servo trabalha.

WebServer na porta 80 — atende 1 conexão por vez

`GET /`	painel de controle
`GET /3d`	modelo 3D (115 KB)
`GET /style.css`	estilo do painel
`GET /app.js`	script do painel
`GET /status`	estado, porta, uptime, RSSI
`GET /sensor`	distância, metal, peso
`GET /atuador`	?estado=on|off — move a portinha

Wi-Fi 2,4 GHz
WPA2 · canal 6
hotspot do celular
(o ESP32 não enxerga 5 GHz)

GET /status, /sensor
a cada 2 s, timeout 3 s
JSON do estado real

GET /atuador?estado=
ao clicar no botão
`{"ok":true,...}`

ACESSO
`http://<IP>/`
O IP aparece no OLED e no Monitor Serial ao ligar a placa.

NAVEGADOR (PC OU CELULAR)

`index.html + app.js`
Painel de controle — rota /

- consulta /status e /sensor a cada 2 s
- mostra estado, porta, sensores e Wi-Fi
- botões Abrir porta / Fechar porta
- o botão fica travado enquanto o comando está em voo
- só avisa o 3D depois que a placa responde {"ok": true}

postMessage (iframe)
BroadcastChannel (outras abas)

`ponto_coleta3d.html`
Modelo 3D (Three.js via CDN) — rota /3d

- entra num iframe ao lado dos cartões (embaixo deles no celular)
- reproduz o que a placa fez: servo, telas do OLED e portinha
- com ?embed=1 não consulta a placa — evita dois clientes na mesma conexão
- aberto sozinho, consulta /status a 500 ms

REGRA PRINCIPAL

O estado verdadeiro é o do ESP32.
O 3D nunca mostra algo que a placa não fez. Sem resposta, não muda nada.

QUANDO A PLACA NÃO RESPONDE
3 falhas seguidas (~6 s) e o 3D cai para simulação, avisando no selo. Volta sozinho.

POR QUE O PAINEL REPASSA O ESTADO
O WebServer atende uma conexão por vez. Uma consulta só, repassada, carrega menos.

`src/main.cpp · src/client/{index.html, app.js, style.css, ponto_coleta3d.html} · src/pagina_web.h gerado por tools/gerar_pagina_web.py (nunca editado à mão)`

Compilação: RAM 14,0% · Flash 72,9%